

CPE 324-03: Advanced Digital Logic Design Laboratory

Lab 6

Temperature Sensor

Submitted by: Hannah Kelley

Date of Experiment: February 24 and 27, 2026, March 3 and 5, 2026

Report Deadline: March 17, 2026

Demonstration Deadline: March 17, 2026

1 Introduction

The purpose of this project is to learn how to interface an external programmable chip with an FPGA system. In this lab, the DS1620 Digital Thermometer and Thermostat temperature sensor was interfaced with the Terasic DE2-115 FPGA Development Board. A Verilog template was modified to implement the communication protocol required by the sensor, allowing the FPGA to initiate temperature conversions and read the resulting data. The interface module was then integrated into the system so that the measured temperature could be displayed on the DE2-115 seven-segment LEDs in either Fahrenheit or Celsius. Prior to hardware testing, the design was verified through simulation using QuestaSim to ensure correct operation.

2 Experiment Description

2.1 Phase 1 Description

The top-level design for this laboratory, given as a structural template, is shown in Figure 1 below. The main focus of phase one is designing the DS1620_INTERFACE module. The module connects into the larger design with trigger and clock inputs and clock, reset, dq, and temperature outputs, as shown in Figure 1.

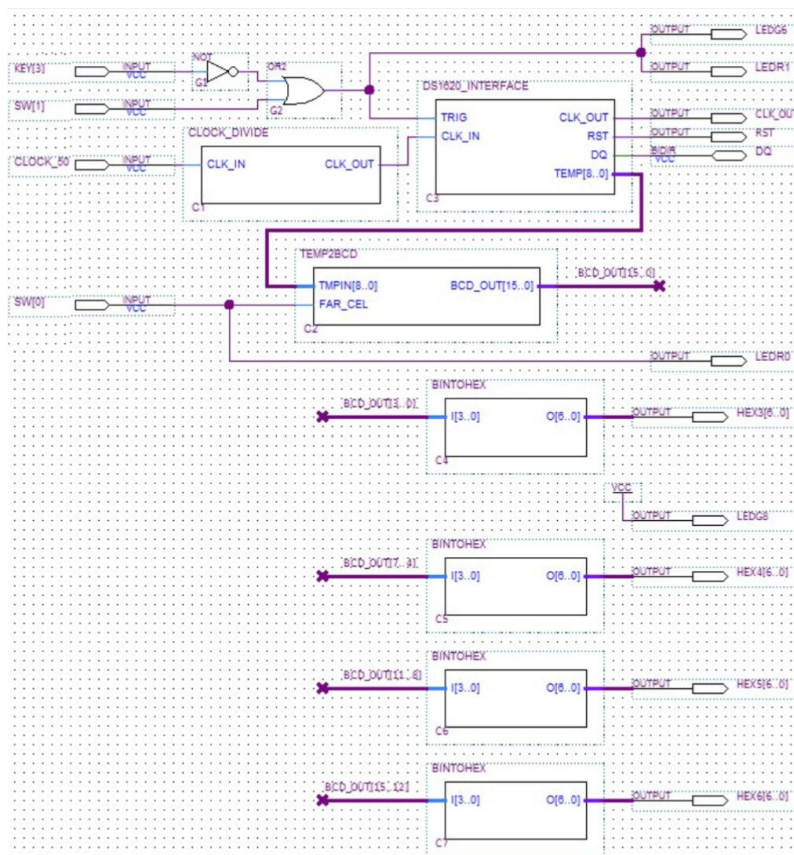


Figure 1: Structural Verilog Schematic

constructing the interface module in Verilog, the suggested waveform in Figure 2 was considered. The waveform outlines the timing and sequence of all signals present in the module,

internal and external. The resulting Verilog code (shown in Appendix A) generates precise clock signals, the reset pulse, and data transmission signals between the greater design and the DS1260.

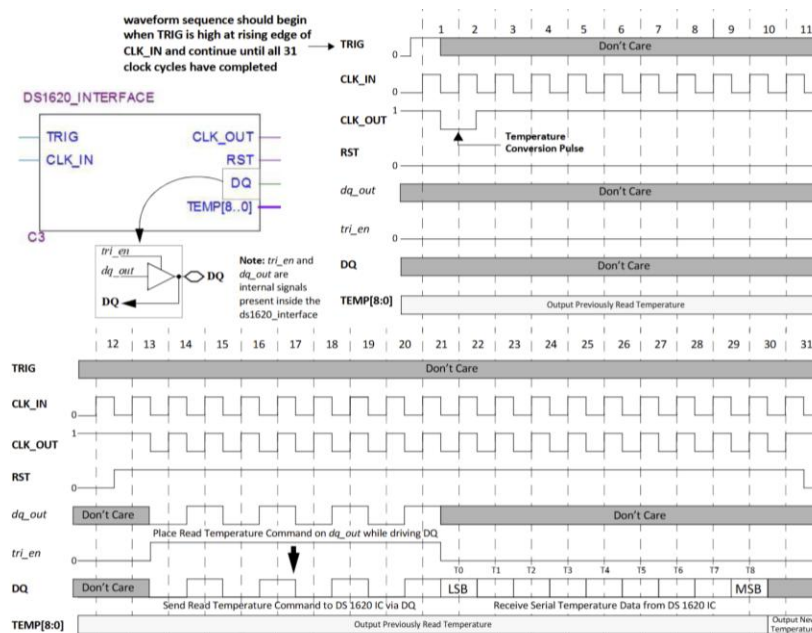


Figure 2: Suggested Waveform Output

The complete DS1620_INTERFACE module was simulated and the resulting waveform was then compared to the expected output given in Figure 2.

2.2 Phase 2 Description

Once the Verilog interface was implemented and simulated to confirm functionality and the correct waveform, the code was implemented on the DE2-115 FPGA. The temperature, in both Fahrenheit and Celsius based on the position of Switch 3, was displayed on the 7-segment LED displays. Additionally, a push button served as the clear signal. The physical wiring and pinouts are given in Figure 3 below.

Signal Name	Cyclone IV E Pin Number	DE2-115 GPIO_0 Pin	UAH ECE DE2-115 Breakout board Pin	DS 1620 Pin Number	DS 1620 Pin Name
VCC		11	3V3	8	VDD
GND		12	GND	4	GND
DQ	PIN_AE20	14	D30	1	DQ
CLK_OUT	PIN_AF20	32	D32	2	CLK/CONV
RST	PIN_AH23	16	D34	3	RST

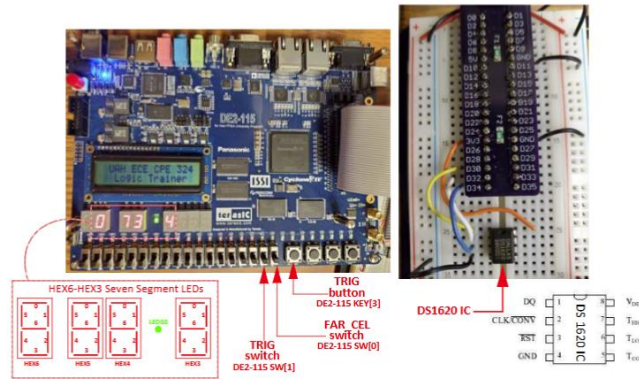


Table 2: 50 Mhz Internal Clock and Switches on the DE2-115 Board

Signal Name	Cyclone IV E Pin Number	Description
CLK_IN	PIN_Y2	50 Mhz clock input
FAR_CEL	PIN_AB28	DE2-115 Toggle Switch SW[0] -- Fahrenheit when down, Celsius when up
TRIG	PIN_R24	Push button Key Switch KEY[3] -- Trigger temp conversions when pressed, stop when released
TRIG	PIN_AC28	DE2-115 Toggle Switch SW[1] -- Trigger temp conversions continuously on when in up position

Table 3: Cyclone IV E Pin Numbers for HEX6-HEX3 7-Seg LEDs on DE2-115 Board

Segment	HEX6		HEX5		HEX4		HEX3	
	Signal Name	Cyclone IV E Pin No.	Signal Name	Cyclone IV E Pin No.	Signal Name	Cyclone IV E Pin No.	Signal Name	Cyclone IV E Pin No.
0	HEX6[0]	PIN_AA17	HEX5[0]	PIN_AD18	HEX4[0]	PIN_AB19	HEX3[0]	PIN_V21
1	HEX6[1]	PIN_AB16	HEX5[1]	PIN_AC18	HEX4[1]	PIN_AA19	HEX3[1]	PIN_U21
2	HEX6[2]	PIN_AA16	HEX5[2]	PIN_AB18	HEX4[2]	PIN_AG21	HEX3[2]	PIN_AB20
3	HEX6[3]	PIN_AB17	HEX5[3]	PIN_AH19	HEX4[3]	PIN_AH21	HEX3[3]	PIN_AA21
4	HEX6[4]	PIN_AB15	HEX5[4]	PIN_AG19	HEX4[4]	PIN_AE19	HEX3[4]	PIN_AD24
5	HEX6[5]	PIN_AA15	HEX5[5]	PIN_AF18	HEX4[5]	PIN_AF19	HEX3[5]	PIN_AF23
6	HEX6[6]	PIN_AC17	HEX5[6]	PIN_AH18	HEX4[6]	PIN_AE18	HEX3[6]	PIN_V19

Figure 3: DE2-115 Temperature Connections and Pinout

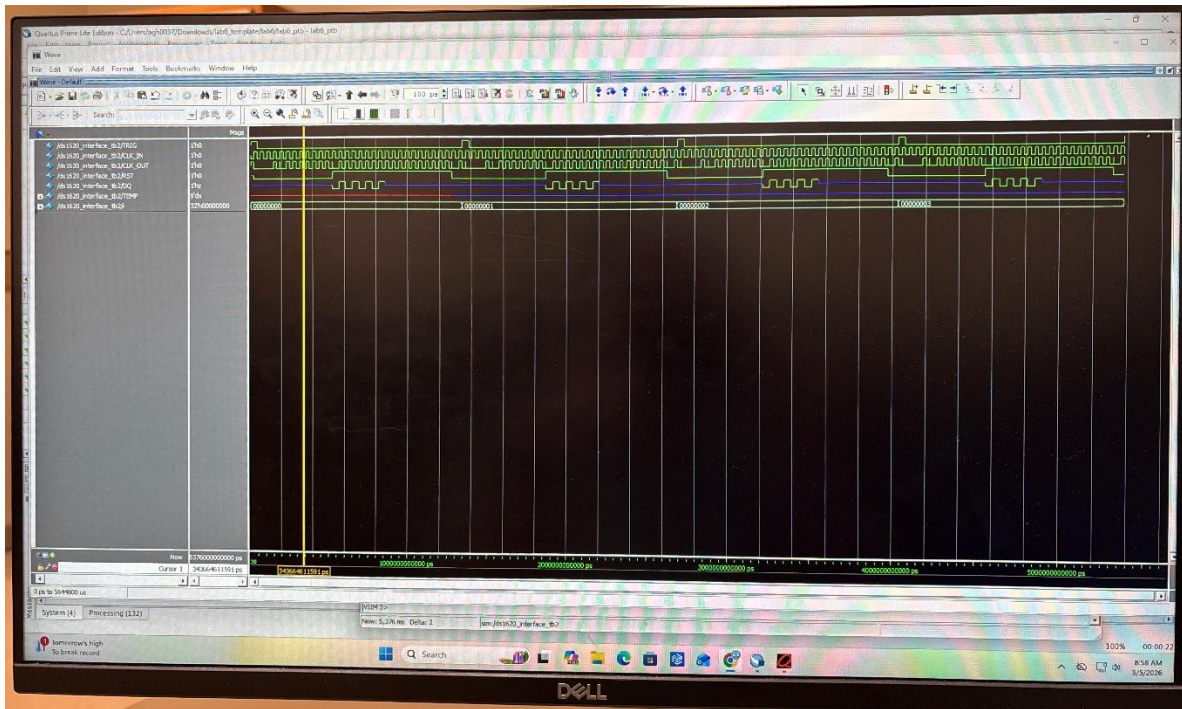
2.3 Phase 3 Description

The final phase of this lab consisted of manually converting the lab6_ptb.v file from VHDL to Verilog. Once this module was converted, it was tested and implemented into the physical testbench. The full code for this module can be found in Appendix C.

3 Results

3.1 Phase 1 Results

After writing the code for the ds1620_Interface.v module (full code in Appendix A), the code was compiled and ran in the Questa simulator. The resulting waveform, shown in Figure 4, was compared to the expected waveform, shown in Figure 2 using varying testbenches. Figure 4 shows the resulting waveform using testbench 2.



3.2 Phase 2 Results

After confirming the working design in simulations, the physical testbench was set up according to the wiring diagram previously shown in Figure 2. The design performed as expected, giving the room temperature readings in both Fahrenheit and Celsius based on the position of the toggle switch (switch 0). The results for both measurements are shown in Figures 5 and 6 below.

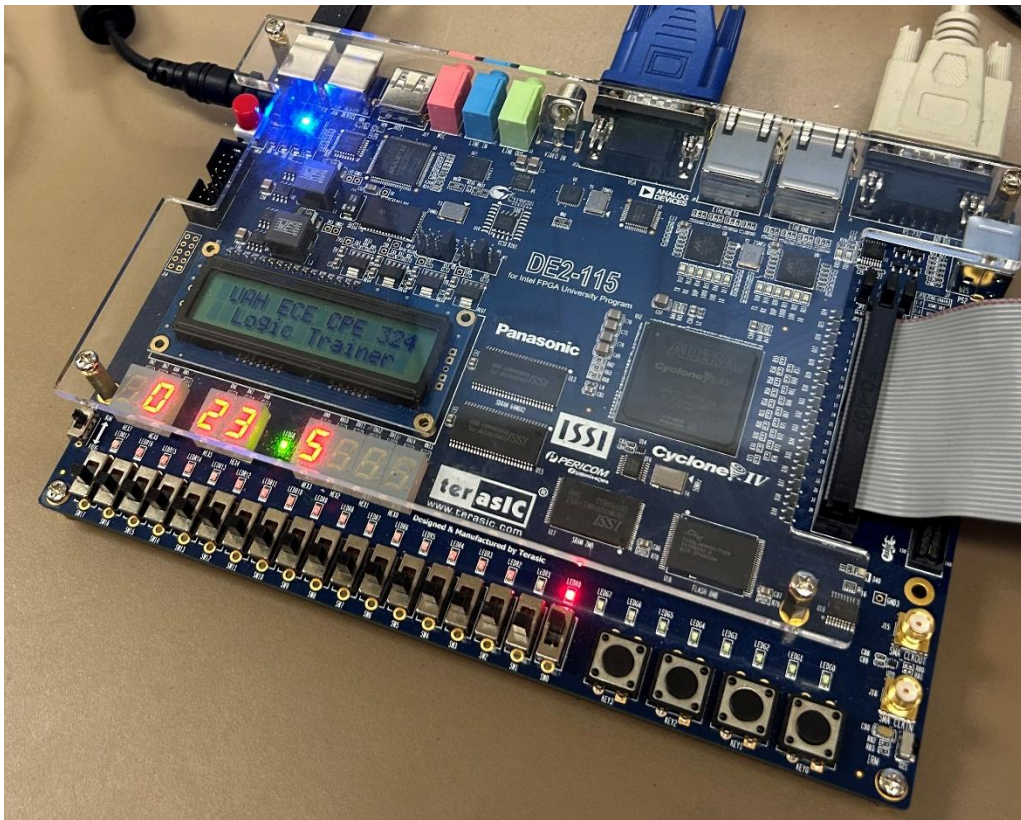


Figure 5: Room Temperature in Celsius

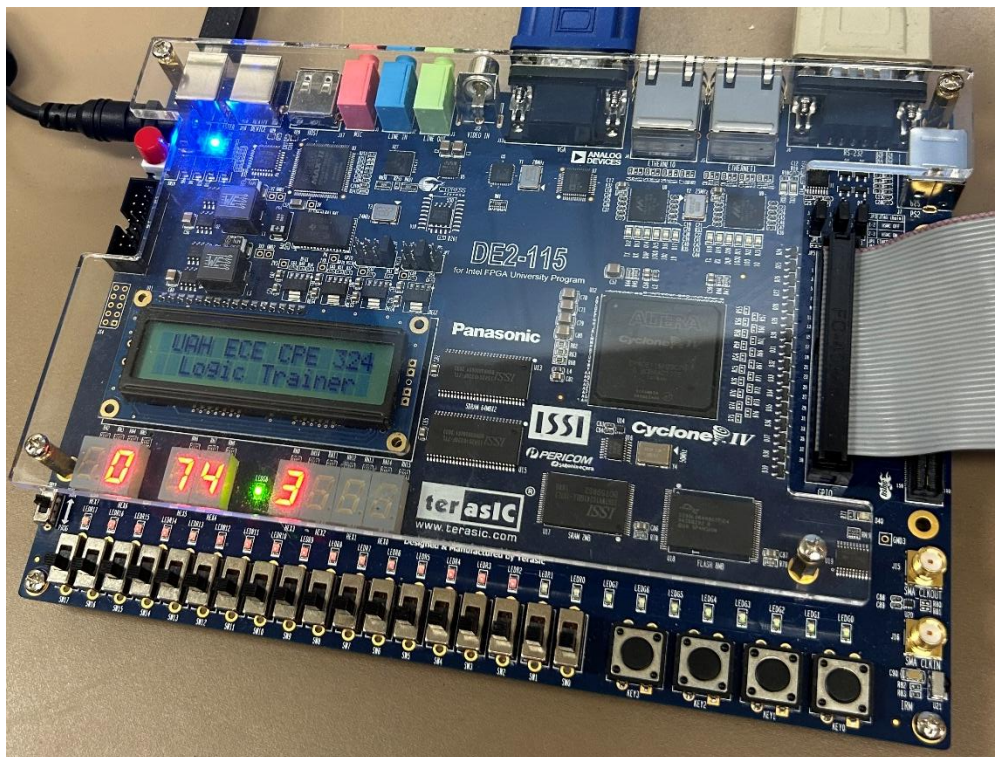


Figure 6: Room Temperature in Fahrenheit

3.3 Phase 3 Results

The final phase of this lab involved converting the lab6_ptb.v file from VHDL to Verilog code and confirming functionality. The full code for this file can be found in Appendix C. Figure 7 shows the simulation results for the rewritten module using testbench 2. The results are nearly identical to the expected results, with only picosecond delays between the two. The near perfect match suggests the two modules are functionally equivalent, which satisfies the aim of this phase.

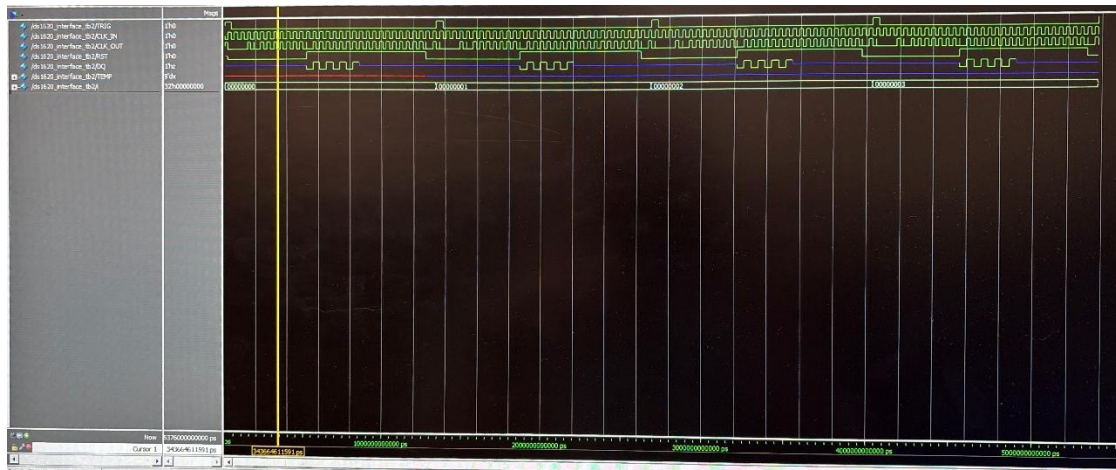


Figure 7: Rewritten Module Output

Additionally, the module was tested on the physical testbench. Again, the results matched the expected. The temperature measurements in Fahrenheit and Celsius can be found in Figures 8 and 9 below. The temperature measurements matched those found in Figures 5 and 6 from the previous physical testbench testing.

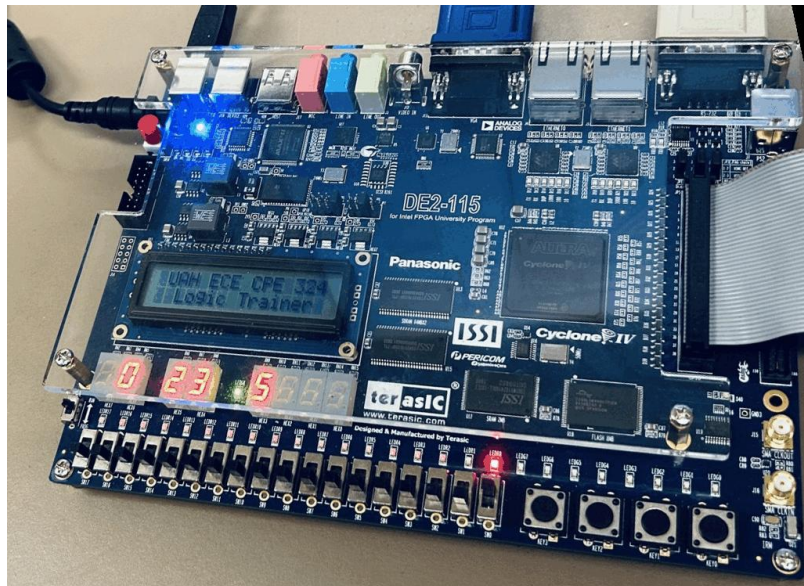


Figure 8: Room Temperature in Celsius

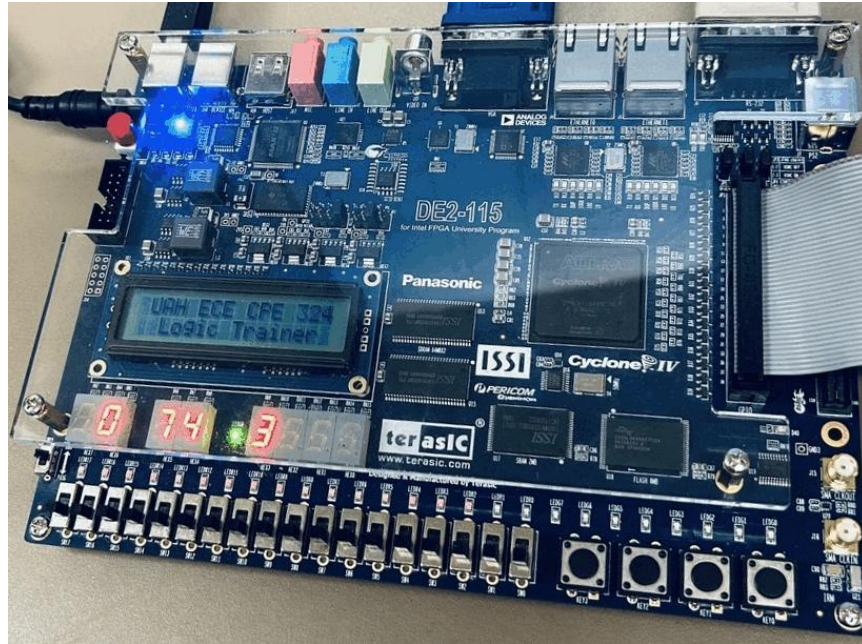


Figure 9: Room Temperature in Fahrenheit

3.4 Post-Lab Questions

After concluding the lab, the following questions were considered:

- 1. In your report describe in detail what each component in the physical testbench does.**

The physical testbench makes use of the DE2-115 FPGA and its components to test the various modules in this project. The button KEY3 allowed for a manual trigger to initiate the temperature reading from the DS1620 temperature sensor. The switch SW1 serves a similar purpose, but instead of a single pulse it allows for continuous triggers to be sent for repeated measurements. The switch SW0 allows the user to toggle between the Celsius and Fahrenheit modes to get measurements in both measurements systems. The clock_divide module is used to slow the FPGA's 50MHz clock signal into a suitable speed for the DS120. The ds1620_interface module then uses this slower clock to drive its reset (RST), clock (CLK), and DQ signals to start a temperature reading and conversion of a measurement. The measurement value is then passed into the temp2bcd module to convert the measurement from binary to BCD and then from Celsius to Fahrenheit, if necessary. Finally, the BCD digits are sent through the binto hex module to drive the seven-segment displays so the measurement can be read easily on the LCDs. The various LEDs on the board serve as status indicators to show when the trigger is active or which temperature measurement system has been selected.

- 2. Rewrite a functionally equivalent CLOCK_DIVIDE module in Verilog so that it that could interface directly with the rest of the testbench design.**

The following module is my version of the CLOCK_DIVIDE module written entirely in Verilog instead of the original VHDL:

```

module CLOCK_DIVIDE (
    input CLK_IN, // 50 MHz in
    output CLK_OUT // slower clock out
);

    reg [20:0] next_clk = 21'd0; // 21 bit counter
    always @(posedge CLK_IN) begin
        next_clk <= next_clk + 1'b1;
    end
    assign CLK_OUT = next_clk[20];

endmodule

```

Table 1: CLOCK_DIVIDE Module in Verilog

4 Conclusion

In this laboratory experiment, a Verilog interface module was developed to communicate with the DS1620 Digital Thermometer and Thermostat and integrated into an FPGA system implemented on the Terasic DE2-115 FPGA Development Board. The project was completed in three phases: simulation-based verification of the ds1620_interface module, hardware verification using the physical testbench, and the manual creation and integration of a Verilog version of the testbench module.

Simulation using QuestaSim was first used to verify the timing and functionality of the interface module before implementing the design on the FPGA. During simulation, debugging the waveform proved challenging due to issues with the bidirectional DQ line, which initially did not appear to drive any output values. This made it difficult to observe the expected communication behavior and required additional investigation of the tri-state control logic before the signal behaved correctly in the simulation environment.

After successful simulation, the module was incorporated into the physical testbench and demonstrated correct operation by communicating with the DS1620 sensor and producing temperature readings on the board. The final phase involved creating and integrating the lab6_ptb.v module, reinforcing the process of module integration within a larger digital design. Overall, the lab provided valuable experience in HDL design, simulation-based debugging, and the practical challenges of interfacing external hardware devices with an FPGA system.

Appendix A – Full Verilog Code for ds1620_Interface.v

```
module ds1620_interface(
    input TRIG, CLK_IN,
    output CLK_OUT, RST,
    output reg [8:0] TEMP,
    inout DQ
);

    reg dq_out;
    reg tri_en = 0;
    reg and_mask = 1, or_mask = 1;
    reg [4:0] cycle_count = 5'd0; // clock cycle count
    reg [8:0] next_temp; // next temperature value
    reg rst;

    // output signal assignments
    assign CLK_OUT = (CLK_IN || or_mask) && and_mask;
    assign RST = rst;
    assign DQ = (tri_en) ? dq_out : 1'bz;

    // rising clock edge signals
    always @(posedge CLK_IN) begin

        // start 31 clock cycle sequence when TRIG is high
        if (TRIG) begin
            cycle_count <= 5'd0;
        end

        if (cycle_count < 5'd31) begin
            cycle_count <= cycle_count + 5'd1;
        end else begin
            cycle_count <= 5'd0;
            TEMP <= next_temp;
        end

        // receive temp bits LSB first
        if (cycle_count >= 5'd22 && cycle_count < 5'd30) begin
            next_temp[cycle_count - 5'd22] <= DQ;
        end

        // or mask
        if (cycle_count >= 5'd13 && cycle_count < 5'd30) begin
            or_mask <= 0;
        end else begin
            or_mask <= 1;
        end
    end
end
```

```

end

// falling clock edge signals
always @(negedge CLK_IN) begin

    // reset
    if (cycle_count >= 5'd12 && cycle_count <= 5'd30) begin
        rst <= 1;
    end else begin
        rst <= 0;
    end

    if (cycle_count == 5'd14 || cycle_count == 5'd16 ||
        cycle_count == 5'd18 || cycle_count == 5'd20) begin
        dq_out <= 1;
    end else begin
        dq_out <= 0;
    end

    // TRI_EN
    if (cycle_count >= 5'd13 && cycle_count < 5'd21) begin
        tri_en <= 1; // send read signal to sensor
    end else begin
        tri_en <= 0;
    end

    // and mask
    if (cycle_count == 5'd1) begin
        and_mask <= 0;
    end else begin
        and_mask <= 1;
    end

end

endmodule

```

Appendix B - Full Verilog Code for clock_divide.v

```

// rewrite of clock_divide module from VHDL to Verilog
// 3/9/2026

module clock_divide (
    input CLK_IN, // 50 MHz in
    output CLK_OUT // slower clock out
);

```

```

    reg [20:0] next_clk = 21'd0; // 21 bit counter

    always @(posedge CLK_IN) begin
        next_clk <= next_clk + 1'b1;
    end

    assign CLK_OUT = next_clk[20];

endmodule

```

Appendix C - Full Verilog Code for lab6_ptb.v

```

// rewrite of lab6_ptb from VHDL to Verilog
// 3/9/26

module lab6_ptb(
    input CLOCK_50,
    input [3:3] KEY,
    input [1:0] SW,
    inout DQ,
    output CLK_OUT,
    output[6:0] HEX3,
    output[6:0] HEX4,
    output[6:0] HEX5,
    output[6:0] HEX6,
    output LEDG8,
    output LEDG6,
    output LEDR0,
    output LEDR1
);

    wire [15:0] BCD_OUT;
    wire [8:0] TEMP;
    wire DS1620_TRIG;
    wire DS1620_CLK;

    // trig asserted by key3 or sw1
    assign DS1620_TRIG = ~KEY[3] || SW[1];

    // LEDs above switches are on when trig is high
    assign LEDR1 = DS1620_TRIG;
    assign LEDG6 = DS1620_TRIG;

    assign LEDR0 = SW[0]; // LED to indicate C/F mode

    assign LEDG8 = 1'b1; // decimal point on LCDs

```

```

// module instantiations
clock_divide C1 (
    .CLK_IN(CLOCK_50),
    .CLK_OUT(DS1620_CLK)
);

temp2bcd C2 (
    .FAR_CEL(SW[0]),
    .TMPIN(TEMP),
    .BCD_OUT(BCD_OUT)
);

ds1620_interface C3 (
    .TRIG(DS1620_TRIG),
    .CLK_IN(DS1620_CLK),
    .CLK_OUT(CLK_OUT),
    .RST(RST),
    .DQ(DQ),
    .TEMP(TEMP)
);

bintoheX C4 (
    .I(BCD_OUT[3:0]),
    .O(HEX3)
);

bintoheX C5 (
    .I(BCD_OUT[7:4]),
    .O(HEX4)
);

bintoheX C6 (
    .I(BCD_OUT[11:8]),
    .O(HEX5)
);

bintoheX C7 (
    .I(BCD_OUT[15:12]),
    .O(HEX6)
);

```

```
endmodule
```